

Arquitetura de Software — ExtraUFLA

1. Visão geral

O ExtraUFLA é estruturado como um **monorepo** com dois aplicativos e dois pacotes compartilhados, orquestrado pelo Turbo. A comunicação entre frontend e backend ocorre exclusivamente via **HTTP REST**.

extraufla/	← raiz do monorepo (Turbo + Bun workspaces)
apps/	
web/	← frontend SPA (React + Vite)
server/	← backend API (NestJS + Node.js)
packages/	
ui/	← componentes compartilhados (shadcn/ui)
config/	← configuração base de TypeScript

A decisão de usar um monorepo facilita a rastreabilidade entre camadas e elimina divergências de versão entre apps e pacotes. O Turbo garante execução paralela de tarefas (**dev**, **build**, **check-types**) com cache incremental.

2. Arquitetura em camadas

A solução é organizada em quatro camadas verticais, presentes tanto no frontend quanto no backend:

APRESENTAÇÃO	React + TanStack Router (apps/web)
APLICAÇÃO	NestJS Controllers / Routes
DOMÍNIO	NestJS Services + Better Auth
PERSISTÊNCIA	Drizzle ORM + libSQL (SQLite/Turso)

Camada	Responsabilidade	Tecnologia
Apresentação	Renderização de UI, rotas do cliente, formulários, feedback visual	React 19, TanStack Router, Tailwind CSS
Aplicação	Recebimento de requisições HTTP, validação de entrada, orquestração	NestJS Controllers (@Controller, @Get, @All)

Camada	Responsabilidade	Tecnologia
Domínio	Regras de negócio, sessão, autenticação, seeding	NestJS Services (<code>@Injectable</code> , <code>OnModuleInit</code>)
Persistência	Schema relacional, migrações, acesso a dados tipado	Drizzle ORM + <code>@libsql/client</code>

Separação frontend / backend

O frontend (`apps/web`) não acessa o banco diretamente. Toda interação ocorre via chamadas HTTP à API em `apps/server`. O único contrato entre as camadas é a interface HTTP: URLs, métodos e formato JSON das respostas.

<code>apps/web</code> (Vite/React, porta 5173)	HTTP REST	<code>apps/server</code> (NestJS/Node.js, porta 3000)
---	-----------	--

3. Backend — `apps/server`

3.1 Estrutura de módulos NestJS

O servidor adota a arquitetura nativa do NestJS: cada área funcional é encapsulada em um **módulo** que agrupa providers, controllers e dependências.

```
apps/server/src/
  main.ts                                # bootstrap: cria app, habilita CORS, listen()
  app.module.ts                          # AppModule: importa os três módulos de domínio
  common/
    env.ts                              # validação de variáveis de ambiente (Zod)
  database/
    database.module.ts                  # DatabaseModule (@Global): provider 'DATABASE'
    schema/
      auth.ts                           # tabelas user, session, account, verification
      course.ts                         # tabela course + relação com user
      index.ts                          # re-exporta todo o schema
  auth/
    auth.module.ts                      # AuthModule: AuthService + AuthController
    auth.service.ts                     # Better Auth: instancia e configura a lib
    auth.controller.ts                  # @All('*') em /api/auth -
  delega ao handler
  courses/
    courses.module.ts                   # CoursesModule: CoursesService + CoursesController
    courses.service.ts                  # seeding idempotente + listCourses()
    courses.controller.ts                # GET /courses
```

3.2 Hierarquia de módulos e injeção de dependências

```
AppModule
  DatabaseModule (@Global)
    provider: 'DATABASE' → DrizzleDB (useFactory)
  AuthModule
    AuthService ← injeta 'DATABASE'
    AuthController
  CoursesModule
    CoursesService ← injeta 'DATABASE'
    CoursesController
```

O `DatabaseModule` é declarado com `@Global()`, de modo que o token `'DATABASE'` é injetável em qualquer módulo sem precisar importar `DatabaseModule` explicitamente. Isso evita acoplamento de importação nos módulos consumidores.

3.3 DatabaseModule — padrão Factory + Facade

```
// database/database.module.ts (simplificado)
const databaseProvider = {
  provide: 'DATABASE',
  useFactory: (): DrizzleDB => {
    const client = createClient({ url: env.DATABASE_URL });
    return drizzle({ client, schema });
  },
};

@Global()
@Module({ providers: [databaseProvider], exports: ['DATABASE'] })
export class DatabaseModule {}
```

O `useFactory` encapsula a construção da instância `Drizzle` (Factory Method). O módulo expõe apenas o token `'DATABASE'` para o restante da aplicação, ocultando `@libsql/client` e os detalhes de configuração (Facade).

3.4 AuthModule — autenticação com Better Auth

O `AuthService` recebe a instância `Drizzle` via injeção de dependências (`@Inject('DATABASE')`) e inicializa o `Better Auth` com o adapter `Drizzle`. O `AuthController` monta um handler curinga em `/api/auth/*` que delega todas as requisições ao handler nativo da lib, incluindo sign-up, sign-in, sign-out e gerenciamento de sessão.

Regras de negócio implementadas no `AuthService`:

- Restrição de cadastro a e-mails `@*.ufla.br` (via `databaseHooks`).
- Configuração de cookies `httpOnly + sameSite: none + secure: true`.

3.5 CoursesModule — seeding e listagem

O `CoursesService` implementa `OnModuleInit`: ao inicializar, executa o seeding idempotente dos cursos de graduação a partir de `data/courses.json` (inserção com `onConflictDoNothing`). O `CoursesController` expõe `GET /courses`.

3.6 Validação de ambiente

O arquivo `common/env.ts` usa `@t3-oss/env-core` com `Zod` para validar e tipar as variáveis de ambiente no bootstrap. Qualquer variável ausente ou inválida causa falha imediata (`fail-fast`), com mensagem descritiva.

```
DATABASE_URL      STRING (min 1)
BETTER_AUTH_SECRET STRING (min 32)
BETTER_AUTH_URL    URL
CORS_ORIGIN        URL
NODE_ENV           enum(development|production|test) default: development
PORT              number default: 3000
```

3.7 Schema de banco de dados

O schema relacional é declarado com `Drizzle ORM` em `database/schema/`:

```
user          id(PK), name, email(unique), emailVerified, image, courseId(FK→course)
session       id(PK), expiresAt, token(unique), userId(FK→user), ipAddress, userAgent
account       id(PK), accountId, providerId, userId(FK→user), accessToken, ...
verification  id(PK), identifier, value, expiresAt
course        id(PK), name(unique), createdAt, updatedAt
```

A relação `user → course` (N:1) suporta a personalização de conteúdo por curso (RF03, RF04), central na proposta do ExtraUFLA.

4. Frontend — apps/web

4.1 Estrutura de arquivos

```
apps/web/src/
  main.tsx          # entrada: TanStack Router + ThemeProvider
  index.css          # Tailwind CSS base
  routes/
    __root.tsx       # layout raiz (Header, Toaster, Devtools)
    index.tsx         # página inicial / landing
    login.tsx         # página de login/cadastro
    dashboard.tsx     # área autenticada
  components/
    header.tsx        # navbar com ThemeToggle e UserMenu
    sign-in-form.tsx  # formulário de login (TanStack Form + Zod)
```

sign-up-form.tsx	# formulário de cadastro
user-menu.tsx	# dropdown com info do usuário e sign-out
mode-toggle.tsx	# botão dark/light mode
theme-provider.tsx	# next-themes provider
loader.tsx	# spinner de carregamento
lib/	
auth-client.ts	# Better Auth client (aponta para VITE_SERVER_URL)
env.ts	# validação de VITE_SERVER_URL (Zod)

4.2 Roteamento e guardas

O TanStack Router usa **roteamento baseado em arquivos** (routes/). Rotas protegidas redirecionam para /login quando não há sessão ativa. A sessão é gerenciada pelo auth-client do Better Auth, que mantém o estado no cliente via cookies.

4.3 Formulários e validação

Formulários usam @tanstack/react-form + resolvers Zod para validação de campos (e-mail institucional @*.ufla.br, senha, nome). Erros são exibidos inline; toasts do sonner notificam sucesso/falha das operações.

5. Pacotes compartilhados

Pacote	Conteúdo	Consumidores
packages/ui	Componentes reutilizáveis baseados em shadcn/ui (Button, Input, Card, Form, etc.)	apps/web
packages/config	Configuração base de TypeScript (tsconfig.base.json)	apps/web, apps/server

6. Diagrama de componentes

```
graph TD
  subgraph Monorepo
    subgraph "apps/web (React + Vite)"
      W_Routes["routes/ (TanStack Router)"]
      W_Components["components/"]
      W_Lib["lib/ (auth-client, env)"]
    end
  end
```

```

subgraph "apps/server (NestJS + Node.js)"
  S_Main["main.ts (bootstrap)"]
  S_App["AppModule"]
  subgraph "DatabaseModule @Global"
    DB_Provider["provider: 'DATABASE'\n(DrizzleDB)"]
  end
  subgraph "AuthModule"
    Auth_Svc["AuthService\n(Better Auth)"]
    Auth_Ctrl["AuthController\n/api/auth/*"]
  end
  subgraph "CoursesModule"
    Courses_Svc["CoursesService\n(seeder + list)"]
    Courses_Ctrl["CoursesController\nGET /courses"]
  end
  S_DB["libSQL / SQLite"]
end

subgraph "packages/"
  UI["packages/ui\n(shadcn components)"]
  CFG["packages/config\n(tsconfig base)"]
end

W_Routes -- "HTTP REST" --> S_Main
W_Components --> W_Routes
W_Lib --> W_Routes
W_Routes --> UI

S_Main --> S_App
S_App --> DB_Provider
S_App --> Auth_Svc
S_App --> Courses_Svc
Auth_Svc -- "@Inject('DATABASE')" --> DB_Provider
Courses_Svc -- "@Inject('DATABASE')" --> DB_Provider
Auth_Ctrl --> Auth_Svc
Courses_Ctrl --> Courses_Svc
DB_Provider --> S_DB

```

7. Fluxo de autenticação

```

sequenceDiagram
  participant Browser
  participant Web as apps/web

```

```

participant Server as apps/server
participant DB as SQLite

Browser->>Web: acessa /login
Web->>Browser: exhibe SignUpForm
Browser->>Web: submete {name, email, password}
Web->>Server: POST /api/auth/sign-up/email
Server->>Server: AuthService valida email @*.ufla.br
Server->>DB: INSERT user + account
DB-->>Server: OK
Server-->>Web: Set-Cookie: session token
Web->>Browser: redireciona para /dashboard

```

8. Relação com requisitos não funcionais

RNF	Descrição	Como a arquitetura atende
RNF01	Desempenho: 2s por requisição	SQLite local, sem round-trip de rede no banco; Drizzle sem N+1
RNF02	Segurança: senhas protegidas, sessão segura	Better Auth: hash bcrypt, cookies httpOnly + secure + sameSite:none
RNF03	Acessibilidade: conformidade básica WCAG	shadcn/ui com roles ARIA nativos; Tailwind sem sobrescrever semântica HTML
RNF04	Manutenibilidade: código legível e modular	NestJS Module/Controller/Service; Biome para lint/format; TypeScript estrito
RNF05	Escalabilidade: base para evolução sem refactor total	Package-by-feature; DI desacopla consumidores de implementações
RNF06	Compatibilidade: navegadores modernos	Vite com targets ES2020+; sem polyfills legados
RNF07	Rastreabilidade: requisitos ligados ao código	Módulos NestJS mapeados a RFs; schema de BD derivado dos requisitos de domínio

9. Decisões arquiteturais relevantes

Decisão	Alternativas consideradas	Justificativa
NestJS como framework backend	Express (sem DI), Fastify+NestJS, Hono	Módulos, DI nativa e decoradores tornam a arquitetura explícita, testável e documentável
Monorepo (Bun workspaces + Turbo)	Repositórios separados, nx	Rastreabilidade entre camadas, build incremental com cache, versões alinhadas
SQLite/libSQL com Drizzle	PostgreSQL, MongoDB	Setup simples, zero custo operacional na fase acadêmica; migração para Turso em produção sem mudar ORM
Better Auth com adapter Drizzle	Auth.js, Clerk, implementação manual	Integração direta com o schema Drizzle, sem duplicar tabelas; regras de domínio (e-mail UFLA) via hooks
TanStack Router (tipado)	React Router, Next.js	Rotas com tipos inferidos, guardas de autenticação na definição de rota, sem overhead de SSR
@Global() em DatabaseModule	Importar DatabaseModule em cada módulo	Evita acoplamento de importação; o banco é um serviço transversal

10. Evolução arquitetural planejada

À medida que os RFs de catálogo (RF05-RF09) e processos seletivos (RF11-RF12) forem implementados, espera-se:

- Introdução do padrão **Repository** em módulos com consultas compostas (filtros, busca, paginação).
- Criação dos módulos **OrganizationModule** e **SelectionProcessModule** seguindo a mesma estrutura package-by-feature.

- Extensão do `CoursesModule` para suportar vínculo entre cursos e organizações (RF04-RF06).
- Possível extração do serviço de autenticação para um módulo separado de políticas, se as regras de domínio de acesso crescerem.

Sprint 06

1. Identificacao

- Numero da sprint: 6
- Período: 16/05/2026 – 23/05/2026
- Data da entrega: 23/05/2026
- Proprietário do produto: Gustavo Dantas
- Scrum Master: Carolina Ude
- Time de desenvolvimento: Angelo Alvarenga, Pedro Martins, Thales Maia

2. Objetivo da sprint

Definir e documentar a arquitetura de software do ExtraUFLA, formalizando camadas, módulos, serviços e decisões de projeto — e executar a migração arquitetural do backend de Express (Bun) para NestJS (Node.js), tornando a arquitetura explícita, diagramável e alinhada ao documento de princípios produzido na Sprint 4.

3. Itens do Sprint Backlog

ID	Item	Responsável	Status
SB01	Migrar <code>apps/server</code> de Express/Bun para NestJS/Node.js	Equipe	Concluído
SB02	Definir e documentar os módulos NestJS (<code>DatabaseModule</code> , <code>AuthModule</code> , <code>CoursesModule</code>)	Gustavo Dantas	Concluído

ID	Item	Responsável	Status
SB03	Absorver <code>packages/db</code> , <code>packages/auth</code> e <code>packages/env</code> como módulos internos de <code>apps/server</code>	Equipe	Concluído
SB04	Restaurar script <code>scrape:courses</code> e migrar execução para Node <code>--experimental-strip-types</code>	Equipe	Concluído
SB05	Preencher <code>docs/arquitetura/arquitetura.md</code> com visão arquitetural, diagramas e relação com RNFs	Gustavo Dantas	Concluído
SB06	Atualizar <code>docs/projeto/decisoes-de-projeto.md</code> com Decisão 3 (NestJS)	Gustavo Dantas	Concluído
SB07	Atualizar <code>docs/padroes/padroes-de-projeto.md</code> com evidências de código NestJS	Gustavo Dantas	Concluído
SB08	Redigir relatório <code>docs/sprints/sprint-06.md</code>	Gustavo Dantas	Concluído
SB09	Atualizar entregas incrementais	Equipe	Concluído

4. Relacao com o conteudo da disciplina

Esta sprint cobre o tópico de **Arquitetura de Software**, com aplicação prática de:

- **Arquitetura em camadas:** separação explícita de apresentação, aplicação, domínio e persistência, tanto no frontend (React/TanStack Router) quanto no backend (NestJS Module/Controller/Service).
- **Decomposição modular:** cada área funcional do backend é encapsulada em um módulo NestJS com responsabilidade única (`DatabaseModule`,

AuthModule, CoursesModule).

- **Injeção de dependências:** uso do container DI nativo do NestJS para desacoplar consumidores de suas implementações concretas (token 'DATABASE', @Inject, useFactory).
- **Documentação arquitetural:** visão estrutural do sistema, diagramas de componentes e sequência, relação com requisitos não funcionais (RNF01-RNF07) e justificativas de decisões arquiteturais.
- **Rastreabilidade:** mapeamento de módulos a requisitos funcionais (RF01-RF15) e não funcionais, conforme estabelecido nas sprints anteriores.

5. Artefatos produzidos

- docs/arquitetura/arquitetura.md — documento principal da sprint: visão geral, arquitetura em camadas, módulos NestJS, schema de BD, diagrama de componentes (Mermaid), diagrama de sequência de autenticação, relação com RNFs e decisões arquiteturais.
- docs/projeto/deciso-es-de-projeto.md — adicionada Decisão 3 (NestJS como framework backend) e decomposição modular atualizada para refletir os módulos NestJS.
- docs/padroes/padroes-de-projeto.md — exemplos de código atualizados para NestJS: @Controller, @Injectable, @Module, useFactory como Factory Method, DatabaseModule e AuthModule como Facades.
- docs/09_entregas_incrementais.md — Sprint 6 marcada como Concluída com resumo da sprint.
- README.md — documentação de pré-requisitos, scripts e estrutura atualizada para refletir NestJS e Node.js.
- apps/server/ — código migrado: main.ts, app.module.ts, database/database.module.ts, auth/auth.module.ts, auth/auth.service.ts, auth/auth.controller.ts, courses/courses.module.ts, courses/courses.service.ts, courses/courses.controller.ts, common/env.ts.
- apps/server/scripts/populate-courses.ts — restaurado e adaptado para execução com `node --experimental-strip-types`.

6. Links e rastreabilidade

- Branch: feature/sprint-06
- PR de código (migração NestJS): #75 (refactor/nestjs-migration)
- Documento de arquitetura: docs/arquitetura/arquitetura.md
- Decisões de projeto atualizadas: docs/projeto/deciso-es-de-projeto.md
- Issues GitHub relacionadas: migração de backend, consolidação do monorepo.

7. Evolucao da aplicacao web

A principal evolução desta sprint foi arquitetural: substituição do servidor Express/Bun por um servidor NestJS/Node.js com injeção de dependências e mó-

dulos explícitos.

Antes (Sprint 5):

```
apps/server/src/
  index.ts          # bootstrap + CORS + rotas inline
  modules/course/   # module/controller/service (Express Router)
packages/
  db/               # schema Drizzle e cliente libSQL
  auth/             # instância Better Auth
  env/              # validação de variáveis
```

Depois (Sprint 6):

```
apps/server/src/
  main.ts           # bootstrap NestJS, CORS
  app.module.ts     # raiz: importa Database, Auth, Courses
  common/env.ts     # validação env (t3-oss/env-core + Zod)
  database/         # DatabaseModule @Global + schema
  auth/             # AuthModule, AuthService, AuthController
  courses/          # CoursesModule, CoursesService, CoursesController
(packages/db, packages/auth, packages/env removidos)
```

Funcionalidades ativas ao fim da sprint:

- RF01: cadastro de usuário com e-mail @*.ufla.br
- RF02: login e gerenciamento de sessão
- RF03: listagem de cursos via GET /courses (base de backend)

8. Dificuldades encontradas

- **Remoção acidental de artefatos.** O gerador do NestJS CLI sobre-escreveu a estrutura de scripts existente, incluindo `populate-courses.ts`. O arquivo foi recuperado do histórico git (`git show HEAD~1:...`) e restaurado.
- **Conflito de gerenciamento de pacotes.** O scaffold NestJS gerou arquivos `pnpm-lock.yaml` e `pnpm-workspace.yaml`, incompatíveis com o Bun workspace da raiz. Todos foram removidos manualmente.
- **Adaptação do script de scraping.** O script `scrape:courses` usava `tsx` para execução TypeScript; foi migrado para `node --experimental-strip-types` (Node >= 22.6), evitando uma dependência adicional no projeto.
- **Decoradores e CommonJS no NestJS.** O NestJS usa `emitDecoratorMetadata` para DI em runtime, incompatível com `--experimental-strip-types`. O comando `nest start --watch` (via SWC internamente) resolve isso sem necessidade de configuração adicional.

9. Revisao do incremento

- O que foi concluído:

- Migração completa de `apps/server` para NestJS com Node.js.
- Três módulos NestJS implementados e funcionais: `DatabaseModule`, `AuthModule`, `CoursesModule`.
- Pacotes separados (`packages/db`, `packages/auth`, `packages/env`) absorvidos e removidos do monorepo.
- Documento de arquitetura `docs/arquitetura/arquitetura.md` preenchido com visão em camadas, diagramas Mermaid, relação com RNFs e decisões arquiteturais.
- Documentação técnica das sprints anteriores atualizada para refletir NestJS.
- README atualizado com pré-requisitos, scripts e estrutura corretos.
- Todos os endpoints existentes preservados e funcionais após a migração.
- O que ficou pendente:
 - Implementação dos RFs de catálogo (RF05-RF07: organizações, detalhes, filtros).
 - Implementação de RF11-RF12 (processos seletivos).
 - Testes automatizados (planejados para Sprint 7).
 - Frontend ainda não consome `GET /courses` (UI de seleção de curso prevista para próximas sprints).

10. Pendências para a próxima sprint

- Definir e documentar o **plano de testes** (Sprint 7): unitários, de integração e end-to-end; matriz de rastreabilidade de testes × requisitos.
- Avaliar introdução do padrão **Repository** quando surgirem consultas compostas (RF05-RF07: filtros e busca de organizações).
- Avançar na implementação dos requisitos de catálogo para viabilizar o fluxo completo de UI.

11. Quadro Kanban

A fazer	Em andamento	Concluído
RF05-RF07 (catálogo, filtros, busca)	—	SB01 Migrar server para NestJS
RF11-RF12 (processo seletivo)	—	SB02 Módulos DatabaseModule, AuthModule, CoursesModule
Testes automatizados (Sprint 7)	—	SB03 Absorver packages/db, auth, env
UI de seleção de curso (frontend)	—	SB04 Restaurar script scrape:courses
—	—	SB05 <code>arquitetura.md</code> preenchido

A fazer	Em andamento	Concluído
—	—	SB06 decisoões-de-projeto.md atualizado
—	—	SB07 padroes-de-projeto.md atualizado
—	—	SB08 sprint-06.md redigido
—	—	SB09 Entregas incrementais atualizadas